

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC QUỐC TẾ



BÁO CÁO ĐỒ ÁN

MÔN: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Dự án: *Plants vs Zombies: Golden Spatula* (Greenfoot Framework)

Tác giả: Lê Anh Kiệt

Giảng viên: L.D.Tân, N.Q.Phú

Ngày 11 tháng 5 năm 2026

Thành viên nhóm

STT	Họ và Tên	MSSV	Vai trò	Trách nhiệm chính
1	Lê Anh Kiệt	ITCSIU24047	All	
2	Dương Gia Thực	ITITWE23026	Zombie	Xây dựng các lớp con Zombie, stats, hoạt ảnh và spawn wave.
3	Nguyễn Quốc Trung Kiên	ITCSIU24034	Zombie	Xây dựng các lớp con Plant, cơ chế đạn và đặt cây trên lưới.
4	Lê Thanh Hoàng	ITCSIU24034	UI	Xây dựng các lớp con Plant, chi phí/hồi chiêu và hoạt ảnh.
5	Nguyễn Viết Huy	ITCSIU24034		

Mục lục

1	Giới thiệu Dự án	4
2	Nguyên tắc Thiết kế Phần mềm và Design Patterns	5
3	Cơ chế Logic Gameplay Chuyên sâu	6
3.1	Hệ thống Hợp nhất Thực thể (Evolutionary Merge Mechanic)	6
3.2	Kiến trúc Dạn và Hệ thống Sắt thương Phân tầng (Projectile System)	6
4	Kiến trúc các Thực thể Plant (Plant Entities Architecture)	7
4.1	1. Lớp Trừu tượng cốt lõi (Core Abstract Plant)	7
4.2	2. Phân nhóm Thực thể Tính Đa hình (Specialized Plant Groups)	7
4.3	3. Sự phối hợp giữa các Thành phần (Delegation Composition)	8
5	Hệ thống Quản lý Tài nguyên và Cấu hình Thực thể	9
5.1	1. Tối ưu hóa bộ nhớ với Static Cache Pattern	9
5.2	2. Quản lý Dữ liệu Thực thể với Enumeration (Data-Driven Approach)	9
5.3	3. Luồng vận hành của thực thể Plant	9
6	Cơ chế Điều phối Kiến trúc Hướng sự kiện (Event-Driven Architecture)	11
6.1	1. Quy trình Hợp nhất thực thể (The Merging Workflow)	11
6.2	2. Hệ thống PlantEventBus Mẫu thiết kế Observer	11
6.3	3. Lợi ích của kiến trúc Loose Coupling	11
7	Hệ thống Khởi tạo Đối tượng (Creational System)	13
7.1	1. Mẫu thiết kế Factory (Factory Pattern)	13
7.2	2. Mẫu thiết kế Singleton (Singleton Pattern)	13
7.3	3. Mối quan hệ và Sự phụ thuộc (Dependencies)	13
8	Hệ thống Điều khiển Hành vi với State Pattern	14
8.1	1. Cấu trúc State Machine	14
8.2	2. Các Trạng thái Thực thi (Concrete States)	14
8.3	3. Cơ chế Chuyển đổi và Điều khiển ngược (Circular Dependency Handling)	14
9	Tổng quan (Overview)	15
10	Phân tích cấu trúc dữ liệu	15
11	Mô hình Toán học và Hình học (Mathematical Modeling)	15
11.1	Tính toán kích thước hình học	15
11.2	Chuyển đổi tọa độ Lưới sang Pixel	15
11.3	Thuật toán xác định vị trí chuột (Inverse Mapping)	16
12	Thuật toán xử lý Logic	16
12.1	Logic Kiểm tra Đặt vật thể (canPlace)	16
12.2	Cơ chế An toàn (returnBackToSafePos)	16
13	Mã nguồn triển khai (Source Code)	16
14	Kết luận	17

1 Giới thiệu Dự án

Dự án mang đến một trải nghiệm hoàn toàn mới lạ bằng cách kết hợp giữa lối chơi thủ thành kinh điển của *Plants vs. Zombies* và phong cách chiến thuật "cờ nhân phẩm" đang thịnh hành từ *Teamfight Tactics (TFT)*. Thay vì cách đặt cây truyền thống, người chơi sẽ tham gia vào một trận chiến đầy biến hóa, nơi sự may mắn và tư duy sắp xếp đội hình được đặt lên hàng đầu.

- **Sự giao thoa độc đáo:** Trò chơi giữ nguyên tính đẹp của các loài cây và thây ma quen thuộc nhưng được vận hành trên một hệ thống bàn cờ lục giác, mở ra vô số cách bài binh bố trận sáng tạo.
- **Cơ chế "Nhân phẩm" kịch tính:**
 - **Cửa hàng ngẫu nhiên:** Người chơi sử dụng tài nguyên *Sun* để "Roll" và săn tìm những loại cây mạnh mẽ hoặc những mảnh ghép còn thiếu cho đội hình.
 - **Nâng cấp sức mạnh:** Khi thu thập đủ các đơn vị giống nhau, hệ thống sẽ tự động thực hiện cơ chế nâng cấp sao, giúp các loại cây thay đổi về cả ngoại hình lẫn chỉ số tấn công.

2 Nguyên tắc Thiết kế Phần mềm và Design Patterns

Để xây dựng một hệ thống game có độ phức tạp cao như "Golden Spatula", dự án đã áp dụng có chủ đích các Mẫu thiết kế (Design Patterns) tiêu chuẩn nhằm tối ưu hóa hiệu suất và đảm bảo khả năng mở rộng (Scalability):

- **Abstract Factory Pattern:** Dự án triển khai interface `IPlantFactory` cùng các lớp cụ thể như `PlantFactory` và `ZombieFactory`. Việc tách biệt logic khởi tạo giúp cô lập quá trình thiết lập thông số phức tạp (`HP`, `Damage`, `Animation`) khỏi logic điều khiển màn chơi. Điều này tuân thủ nguyên lý **Open/Closed**: có thể thêm hàng loạt thực thể mới mà không cần can thiệp vào mã nguồn hiện có.
- **State Pattern (Finite State Machine):** Đây là hạt nhân của hệ thống AI dành cho Zombie. Thông qua interface `IZombieState`, hành vi của thực thể được module hóa thành các lớp trạng thái độc lập (`WalkingState`, `EatingState`, `DeadState`).
 - Mỗi trạng thái tự quản lý vòng đời thông qua phương thức `enter()`, `update()` và `exit()`.
 - Cách tiếp cận này loại bỏ hoàn toàn các cấu trúc `if-else` phức tạp, giúp việc quản lý hoạt ảnh (`Animation`) và logic chuyển đổi trạng thái trở nên minh bạch, dễ bảo trì.
- **Flyweight Pattern:** Nhằm tối ưu hóa tài nguyên trên framework Greenfoot, lớp `SpriteCache` đóng vai trò quản lý tập trung các mảng `GreenfootImage[]`.
 - **Intrinsic State:** Dữ liệu hình ảnh gốc được chia sẻ chung giữa các thực thể cùng loại.
 - **Extrinsic State:** Các trạng thái riêng lẻ như khung hình hiện tại (`frame`), độ trong suốt (`transparency`) được quản lý độc lập bởi mỗi `SpriteAnimator`.
 - Kết quả: Giảm thiểu tối đa dung lượng RAM và hiện tượng giật lag khi có số lượng lớn Entity xuất hiện đồng thời.
- **Observer Pattern:** Trục sự kiện trung tâm `PlantEventBus` triển khai cơ chế "Publish-Subscribe" thông qua interface `IPlantEventListener`. Các hệ thống ngoại vi (`UI`, `Sound`, `Grid Logic`) có thể lắng nghe các biến động như `onPlantDied` hay `onPlantMerged` mà không cần tham chiếu trực tiếp đến đối tượng phát sự kiện, tạo nên một kiến trúc **Loosely Coupled** (phụ thuộc lỏng).
- **Singleton Pattern:** Đảm bảo tính duy nhất và nhất quán của dữ liệu (*Single Source of Truth*) cho các thành phần điều phối toàn cục như `PlantFactory`, `SunManager`, `AudioManager` và `WaveManager`.
- **Interface Segregation (ISP):** Thay vì sử dụng các Interface đa năng, dự án chia nhỏ thành các "hợp đồng" chuyên biệt như `IPlantFactory`, `IPlantEventListener` và `IPlantPlacer`. Điều này đảm bảo các lớp cụ thể chỉ thực thi những phương thức thực sự cần thiết cho vai trò của chúng.

3 Cơ chế Logic Gameplay Chuyên sâu

3.1 Hệ thống Hợp nhất Thực thể (Evolutionary Merge Mechanic)

Cơ chế nâng cấp cấp độ (Tier-up) lấy cảm hứng từ thể loại Auto-battler, vận hành dựa trên sự phối hợp đa luồng giữa hai thành phần:

- **Merger (Chuyển động Đồng bộ):** Khi hệ thống phát hiện điều kiện nâng cấp (3 đơn vị cùng loại và cùng cấp), **Merger** sẽ tạm thời chiếm quyền điều khiển tọa độ của các đơn vị nguồn. Nó thực hiện nội suy chuyển động để đẩy các đơn vị này về phía vị trí đích (**target**). Việc sử dụng các cờ trạng thái **arrived** và **finalized** đóng vai trò như một bộ "Barrier" để đồng bộ hoạt ảnh trước khi thực hiện bước nâng cấp logic cuối cùng.
- **UpgradeManager (Xử lý Logic):** Đóng vai trò là bộ não điều phối. Sau khi nhận tín hiệu hoàn tất từ **Merger**, lớp này sẽ thực hiện quy trình: xóa bỏ các thực thể cũ, truy vấn **IPlantFactory** để sinh ra thực thể mới có chỉ số cao hơn (Ví dụ: **Peashooter** → **Repeater** → **GatlingPea**). Quy trình này đảm bảo dữ liệu về thuộc tính luôn được đóng gói chính xác theo từng cấp độ.

3.2 Kiến trúc Dạn và Hệ thống Sắt thương Phân tầng (Projectile System)

Hệ thống vật thể bay (Projectile) được thiết kế theo sơ đồ kế thừa linh hoạt từ lớp cơ sở **Projectile**, cho phép tái sử dụng logic vật lý và chạm (*Collision Detection*) trong khi vẫn giữ được sự đa dạng về hành vi:

- **Cơ chế Kế thừa và Đa hình:**
 - **Pea:** Dạn tiêu chuẩn của **Peashooter**, xử lý va chạm cơ bản.
 - **FirePea:** Biến thể tăng cường, tích hợp thêm các hiệu ứng sát thương diện rộng hoặc sát thương theo thời gian (*DoT*), thường xuất hiện ở các Plant cấp cao.
 - **Needle (Lv.1, Lv.2, Lv.3):** Hệ thống đạn đặc thù của **Cactus**. Mỗi cấp độ đạn không chỉ thay đổi về hình ảnh mà còn được ánh xạ chỉ số sát thương và hiệu ứng xuyên thấu dựa trên cấp độ sao của Plant thực hiện đòn đánh.
- **Hệ thống PlantDamageType:** Định nghĩa các loại sát thương khác nhau, cho phép mở rộng khả năng tương tác khắc chế giữa các hệ Plant và Zombie trong các bản cập nhật tương lai.

4 Kiến trúc các Thực thể Plant (Plant Entities Architecture)

Dựa trên sơ đồ lớp (UML Diagram), hệ thống thực thể Plant được thiết kế theo cấu trúc phân cấp, cho phép tối ưu hóa việc tái sử dụng mã nguồn và dễ dàng bổ sung các loại thực thể mới mà không ảnh hưởng đến logic cốt lõi.

4.1 1. Lớp Trừu tượng cốt lõi (Core Abstract Plant)

Lớp Plant (kế thừa từ `SpriteAnimator`) đóng vai trò là lớp cơ sở cho tất cả các loại cây trong game. Nó quản lý các thuộc tính và hành vi chung:

- **Quản lý Trạng thái:** Sử dụng biến `state` kiểu `PlantState` để điều phối giữa các hành động `IDLE`, `ATTACKING`, và `DYING`.
- **Đồng bộ Tọa độ:** Phương thức `syncGridPosition()` đảm bảo thực thể luôn nằm đúng trong hệ thống ô lưới (Grid System) của trò chơi ngay cả khi thực hiện các animation phức tạp.
- **Cơ chế Tương tác:** Các phương thức `hit(int dmg)` và `onDeath()` được định nghĩa sẵn để xử lý logic sống còn cơ bản.

4.2 2. Phân nhóm Thực thể Tính Đa hình (Specialized Plant Groups)

Hệ thống chia các lớp con thành các nhóm chức năng riêng biệt, mỗi nhóm ghi đè (override) các phương thức cụ thể để tạo ra đặc tính riêng:

4.2.1 Nhóm Tấn công Tầm xa (Projectile Group)

Đây là nhóm các cây có khả năng bắn đạn, sử dụng logic kiểm tra kẻ thù theo hàng ngang:

- **Peashooter:** Triển khai phương thức `executeShoot()` cơ bản.
- **Repeater:** Ghi đè logic bắn để thực hiện 2 lần bắn trong một chu kỳ hành động.
- **Cactus:** Kết hợp logic kiểm tra hàng (`checkZombieInRow()`) và bắn kim (`fireNeedle()`).

4.2.2 Nhóm Tấn công Cận chiến (Melee Group)

- **BonkChoy:** Khác với nhóm bắn xa, lớp này tập trung vào `handleCombat()` và `applyDmg()` trong phạm vi hẹp quanh thực thể. Cấu trúc lớp `BonkChoy2`, `BonkChoy3` cho thấy khả năng mở rộng cấp độ (tier) dễ dàng.

4.2.3 Cơ chế Nâng cấp Đặc biệt (Evolutionary Tier)

- **GatlingPea:** Đây là lớp nâng cấp bậc cao (Tier 3). Nó giới thiệu thuộc tính `burstCount` để xử lý loạt đạn liên tục (4-5 viên mỗi đợt), minh họa cho việc mở rộng thuộc tính mà không làm phá vỡ cấu trúc của lớp cha Plant.

4.3 3. Sự phối hợp giữa các Thành phần (Delegation Composition)

Sơ đồ UML thể hiện mối quan hệ **Composition** (Thành phần) chặt chẽ giữa **Plant** và các lớp hỗ trợ:

- **PlantStateManager**: Tách biệt logic điều khiển trạng thái ra khỏi logic hình ảnh.
- **PlantInputHandler**: Đảm nhận riêng phần tương tác chuột (`handleMouse()`), giúp lớp **Plant** tập trung hoàn toàn vào gameplay.
- **PlantCombatHandler**: Chuyên trách xử lý các va chạm và tính toán sát thương, tăng tính module hóa cho hệ thống.

Nhận xét kỹ thuật: Việc thiết kế lớp con như **Repeater** hay **GatlingPea** chỉ tập trung ghi đè phương thức `executeShoot()` là minh chứng cho việc áp dụng thành công nguyên lý **Open/Closed Principle** (Mở rộng tính năng nhưng đóng việc chỉnh sửa mã nguồn cốt lõi).

5 Hệ thống Quản lý Tài nguyên và Cấu hình Thực thể

Một trong những thách thức lớn nhất khi phát triển game trên framework Greenfoot là quản lý tài nguyên hình ảnh và cấu hình thuộc tính cho số lượng lớn thực thể. Hệ thống đã giải quyết vấn đề này thông qua hai thành phần cốt lõi: `SpriteCache` và `PlantType`.

5.1 1. Tối ưu hóa bộ nhớ với Static Cache Pattern

Thay vì để mỗi thực thể `Plant` tự quản lý và nạp lại hình ảnh từ bộ nhớ máy tính (Disk I/O), hệ thống sử dụng một cơ chế bộ nhớ đệm tĩnh:

- **SpriteCache:** Sử dụng một cấu trúc dữ liệu `Map<String, GreenfootImage[]>` để lưu trữ tập trung toàn bộ các mảng ảnh animation.
- **Cơ chế hoạt động:** Lớp `SpriteAnimator` giao tiếp với `SpriteCache` thông qua các phương thức `getSprites()`. Nếu mảng ảnh đã tồn tại trong Cache, nó sẽ được trả về ngay lập tức; nếu chưa, hệ thống mới tiến hành nạp ảnh và lưu vào Cache cho các lần sử dụng sau.
- **Lợi ích:** Giảm thiểu tối đa dung lượng RAM tiêu thụ và loại bỏ hiện tượng "giật" (stuttering) khi sinh ra (spawn) các loại cây mới trong quá trình chơi.

5.2 2. Quản lý Dữ liệu Thực thể với Enumeration (Data-Driven Approach)

Thay vì "hard-code" các chỉ số như HP, Damage, hay Cost vào từng lớp cụ thể, hệ thống sử dụng `PlantType` (Enumeration) như một trung tâm cấu hình dữ liệu:

- **Cấu hình tập trung:** Tất cả thông số của `PEASHOOTER`, `REPEATER`, `BONK_CHOY`,... bao gồm giá tiền (`cost`), lượng máu (`hp`), sát thương (`damage`) và tốc độ đánh (`shootDelay`) đều được định nghĩa tại đây.
- **Tính linh hoạt:** Khi cần cân bằng lại trò chơi (Game Balancing), lập trình viên chỉ cần chỉnh sửa các hằng số trong `PlantType` thay vì phải tìm kiếm và sửa đổi trong từng lớp logic riêng biệt.

5.3 3. Luồng vận hành của thực thể Plant

Sự phối hợp giữa các thành phần được thể hiện qua luồng xử lý sau:

1. **Khởi tạo:** Lớp trừa tượng `Plant` chứa một tham chiếu đến `PlantType` để xác định "danh tính" và thông số của nó.
2. **Hoạt họa:** `SpriteAnimator` điều khiển quá trình hiển thị dựa trên các mảng ảnh được lấy từ `SpriteCache`. Phương thức `animate()` thực hiện việc hoán đổi `GreenfootImage` theo chu kỳ thời gian `lastMillis`.
3. **Hiệu ứng:** Phương thức `hitFlash(String)` và `applyImageWithOpacity()` cho phép thay đổi diện mạo thực thể (ví dụ: nháy trắng khi bị thương) một cách linh hoạt mà không làm hỏng dữ liệu ảnh gốc trong Cache.

Đánh giá kỹ thuật: Việc tách biệt hoàn toàn giữa **Dữ liệu cấu hình** (PlantType), **Bộ nhớ đệm** (SpriteCache) và **Logic hiển thị** (SpriteAnimator) giúp hệ thống đạt được độ ổn định cao, dễ dàng bảo trì và mở rộng thêm hàng trăm loại thực thể mới chỉ bằng cách bổ sung cấu hình.

6 Cơ chế Điều phối Kiến trúc Hướng sự kiện (Event-Driven Architecture)

Để vận hành một hệ thống Auto-battler phức tạp với cơ chế nâng cấp tự động, dự án áp dụng mô hình liên lạc gián tiếp thông qua một Trục sự kiện (Event Bus), giúp các thành phần hệ thống hoạt động độc lập và giảm thiểu sự phụ thuộc.

6.1 1. Quy trình Hợp nhất thực thể (The Merging Workflow)

Cơ chế "ghép sao" (Merge) được điều phối bởi sự phối hợp giữa ba thành phần chính:

- **Merger:** Đóng vai trò là một bộ điều khiển hành trình. Khi xác định có hai thực thể cùng loại, **Merger** sẽ "chiếm quyền" (controls) và thực hiện việc di chuyển thực thể **source** về phía **target**. Quá trình này được cập nhật liên tục qua hàm `update()` trả về giá trị `boolean` để báo hiệu trạng thái kết thúc.
- **UpgradeManager:** Đóng vai trò là bộ não logic. Khi **Merger** hoàn tất việc di chuyển, **UpgradeManager** sẽ thực hiện lệnh nâng cấp (calls `static upgrade`) để tính toán loại thực thể mới sẽ xuất hiện (ví dụ: cấp 1 + cấp 1 = cấp 2).
- **Thao tác trên Scene:** Quá trình này diễn ra trực tiếp trên **PlayScene** thông qua việc xóa bỏ các thực thể cũ và thêm thực thể mới vào hệ thống ô lưới (**GridManager**).

6.2 2. Hệ thống PlantEventBus Mẫu thiết kế Observer

Nhằm mục đích thông báo các thay đổi quan trọng trong gameplay cho UI và các hệ thống quản lý khác, dự án triển khai mẫu thiết kế Observer thông qua **PlantEventBus**:

- **IPlantEventListener:** Một Interface định nghĩa các "hợp đồng" sự kiện như `onPlantDied`, `onPlantHit`, `onPlantMerged`. Bất kỳ hệ thống nào muốn nhận tin (như Hệ thống âm thanh, UI thanh máu, Hệ thống tính điểm) đều phải triển khai Interface này.
- **Cơ chế Phát sự kiện (Publishing):** Khi một **Plant** thay đổi trạng thái hoặc bị tiêu diệt, nó không gọi trực tiếp đến UI. Thay vào đó, nó gửi thông điệp tới **PlantEventBus**.
- **Cơ chế Đăng ký (Subscribing):** **PlantEventBus** quản lý một danh sách các `listeners` và sẽ tự động thông báo (broadcast) đến tất cả các đối tượng đã đăng ký khi có sự kiện mới phát sinh.

6.3 3. Lợi ích của kiến trúc Loose Coupling

Việc áp dụng mô hình này mang lại những ưu thế vượt trội về mặt kỹ thuật:

- **Khả năng mở rộng (Scalability):** Dễ dàng thêm các hiệu ứng hình ảnh (VFX) hoặc âm thanh (SFX) mới khi một cây được nâng cấp bằng cách thêm một `Listener` mới vào **PlantEventBus** mà không cần sửa code trong lớp **Plant** hay **Merger**.
- **Dễ dàng gỡ lỗi (Debug):** Các luồng sự kiện được tập trung tại **EventBus**, giúp lập trình viên theo dõi được toàn bộ vòng đời và các tương tác của thực thể một cách minh bạch.

Kết luận kỹ thuật: Sự kết hợp giữa bộ điều phối chuyển động (**Merger**) và trục sự kiện trung tâm (**EventBus**) tạo nên một hệ thống gameplay mượt mà, chuyên nghiệp, phản ánh đúng tư duy thiết kế của các Game Engine hiện đại.

7 Hệ thống Khởi tạo Đối tượng (Creational System)

Trong một trò chơi có nhiều loại thực thể như "Golden Spatula", việc quản lý cách thức và thời điểm tạo ra các đối tượng (Plants, SeedPackets) là cực kỳ quan trọng để đảm bảo tính nhất quán và hiệu suất. Hệ thống đã triển khai sự kết hợp giữa **Abstract Factory** và **Singleton Pattern**.

7.1 1. Mẫu thiết kế Factory (Factory Pattern)

Dự án sử dụng Interface `IPlantFactory` để định nghĩa bộ khung cho việc khởi tạo, giúp tách biệt logic tạo đối tượng ra khỏi logic điều khiển trò chơi:

- **Đa dạng hóa phương thức khởi tạo:** Factory cung cấp khả năng tạo đối tượng thông qua `PlantType` (Enumeration) hoặc qua `String` (`typeName`). Điều này cực kỳ hữu ích khi nạp dữ liệu từ file cấu hình bên ngoài hoặc từ các sự kiện trong game.
- **Tính đóng gói (Encapsulation):** Các lớp như `UpgradeManager` hay `PlayScene` không cần biết chi tiết cách một `Peashooter` hay `GatlingPea` được tạo ra (như nạp ảnh nào, máu bao nhiêu). Chúng chỉ cần yêu cầu Factory: `createPlant(type)`.
- **Quản lý SeedPacket:** Factory không chỉ tạo thực thể trên sân mà còn chịu trách nhiệm tạo các gói hạt giống (`SeedPacket`), đảm bảo sự đồng bộ giữa loại cây và vật phẩm hỗ trợ tương ứng.

7.2 2. Mẫu thiết kế Singleton (Singleton Pattern)

Lớp `PlantFactory` được triển khai dưới dạng Singleton để đảm bảo toàn bộ hệ thống chỉ sử dụng một bộ điều phối khởi tạo duy nhất:

- **Biến tĩnh instance:** Đảm bảo duy nhất một đối tượng Factory tồn tại trong suốt vòng đời của ứng dụng.
- **Truy cập toàn cục:** Phương thức static `getInstance()` cho phép bất kỳ thành phần nào trong game (từ `UpgradeManager` đến các UI components) đều có thể truy cập dịch vụ tạo đối tượng mà không cần truyền tham chiếu phức tạp.

7.3 3. Mối quan hệ và Sự phụ thuộc (Dependencies)

Sơ đồ UML thể hiện rõ luồng tương tác giữa các thành phần khởi tạo và logic nâng cấp:

- **UpgradeManager và Factory:** `UpgradeManager` phụ thuộc trực tiếp vào `IPlantFactory`. Khi quá trình "Merge" hoàn tất, nó sẽ gọi Factory để tạo ra thực thể cấp cao hơn.
- **Sử dụng tài nguyên:** `PlantFactory` trực tiếp sử dụng `PlantType` để truy xuất các thông số mặc định (`cost`, `hp`, `damage`) trước khi chuyển giao đối tượng cho hệ thống quản lý hoạt họa (`SpriteAnimator`).

Đánh giá kỹ thuật: Việc áp dụng **Dependency Inversion** thông qua Interface `IPlantFactory` giúp hệ thống dễ dàng kiểm thử (Unit Testing) và thay thế logic khởi tạo mà không cần sửa đổi các lớp phụ thuộc. Đây là minh chứng cho một kiến trúc phần mềm linh hoạt và có tính chuyên nghiệp cao.

8 Hệ thống Điều khiển Hành vi với State Pattern

Để quản lý các hành vi đa dạng và phức tạp của thực thể kẻ thù (Zombie), dự án đã triển khai mẫu thiết kế **State (Trạng thái)**. Cơ chế này cho phép một đối tượng thay đổi hành vi của nó khi trạng thái nội bộ thay đổi, giúp mã nguồn trở nên sạch sẽ và dễ dàng mở rộng.

8.1 1. Cấu trúc State Machine

Kiến trúc này bao gồm một Interface trung tâm và các lớp trạng thái cụ thể:

- **IZombieState (Interface):** Định nghĩa các phương thức vòng đời cho một trạng thái bao gồm `enter()` (khởi tạo trạng thái), `update()` (duy trì hành vi) và `exit()` (dọn dẹp trước khi chuyển trạng thái). Phương thức `getAnimation()` đảm bảo mỗi trạng thái hiển thị đúng bộ ảnh hoạt họa tương ứng.
- **Lớp Zombie (Context):** Đóng vai trò điều phối, lưu giữ tham chiếu `currentState` kiểu `IZombieState`. Nó giao phó (delegate) toàn bộ logic xử lý cho đối tượng trạng thái hiện tại.

8.2 2. Các Trạng thái Thực thi (Concrete States)

Hệ thống phân tách rõ rệt ba hành vi cốt lõi của Zombie:

- **WalkingState:** Điều khiển Zombie di chuyển trên sân chơi. Trạng thái này liên tục kiểm tra va chạm với Plant để quyết định khi nào cần chuyển sang trạng thái ăn.
- **EatingState:** Thực thi logic tấn công Plant. Trạng thái này điều khiển phương thức `playEating()` của đối tượng Zombie và gây sát thương theo chu kỳ.
- **DeadState:** Xử lý chuỗi hình ảnh khi Zombie bị tiêu diệt. Sau khi `hasFinishedDeathAnim` trở thành `true`, trạng thái này sẽ gọi hàm `cleanup()` để loại bỏ thực thể khỏi thế giới game một cách an toàn.

8.3 3. Cơ chế Chuyển đổi và Điều khiển ngược (Circular Dependency Handling)

Một điểm đặc biệt trong thiết kế là mối quan hệ hai chiều giữa State và Context:

- **Cơ chế Delegation:** Zombie gọi `currentState.update()` trong mỗi chu kỳ `act()` của `Greenfoot`.
- **Cơ chế Callback:** Các lớp State nắm giữ tham chiếu ngược lại đối tượng Zombie (`references/controls`). Điều này cho phép State tự kích hoạt việc chuyển đổi sang trạng thái mới thông qua hàm `setState()` của Zombie khi điều kiện logic được thỏa mãn (ví dụ: máu về 0 thì chuyển sang `DeadState`).

Đánh giá kỹ thuật: Việc áp dụng **State Pattern** giúp tách biệt hoàn toàn logic di chuyển, tấn công và cái chết. Khi muốn thêm một hành vi mới (như Zombie nhảy qua cây hoặc Zombie sử dụng kỹ năng đặc biệt), lập trình viên chỉ cần tạo thêm một lớp mới thực thi `IZombieState` mà không cần sửa đổi mã nguồn hiện có của các trạng thái khác, tuân thủ nghiêm ngặt nguyên lý **Open/Closed**.

9 Tổng quan (Overview)

Lớp `GridManager` là thành phần cốt lõi điều khiển không gian trong trò chơi. Nó không chỉ đơn thuần quản lý tọa độ mà còn đóng vai trò là một bộ điều phối (Coordinator) giữa tương tác của người chơi, logic thực thể (Plant) và các quy tắc tiến hóa (Combination).

Hệ thống sử dụng cấu trúc lưới lục giác (**Hexagonal Grid**) thay vì lưới ô vuông truyền thống để tăng tính chiến thuật và thẩm mỹ cho trò chơi.

10 Phân tích cấu trúc dữ liệu

`GridManager` kế thừa từ `Actor` và thực thi interface `IPlantPlacer`. Các biến thành phần quan trọng bao gồm:

- `Plant[][] Board`: Một ma trận 2 chiều kích thước 6×9 .
 - *Hàng 0-4*: Khu vực chiến đấu chính.
 - *Hàng 5*: Khu vực dự trữ (Staging Area) dùng để chứa các đơn vị chờ.
- **Thông số lưới**: Bán kính lục giác $R = 40$, số cột 9, số hàng 6.

11 Mô hình Toán học và Hình học (Mathematical Modeling)

Việc triển khai lưới lục giác đòi hỏi các phép tính hình học chính xác để đảm bảo các ô khít nhau. Trò chơi sử dụng kiểu bố trí **Pointy Top Hexagons**.

11.1 Tính toán kích thước hình học

Dựa trên bán kính R , ta có các công số sau:

- Chiều rộng của một lục giác (W): $W = R \times \sqrt{3}$
- Chiều cao của một lục giác (H): $H = 2 \times R$
- Bước nhảy ngang (`STEP_X`): $STEP_X = W$
- Bước nhảy dọc (`STEP_Y`): Để các hàng so le khít nhau, khoảng cách tâm giữa 2 hàng chỉ bằng $3/4$ chiều cao: $STEP_Y = H \times 0.75 = 1.5R$.

11.2 Chuyển đổi tọa độ Lưới sang Pixel

Tọa độ Pixel (x_p, y_p) được xác định từ tọa độ lưới (col, row) theo công thức:

$$x_p = \text{ORIGIN_X} + (col \times W) + \left((row \bmod 2) \times \frac{W}{2} \right) \quad (1)$$

$$y_p = \text{ORIGIN_Y} + (row \times 1.5R) \quad (2)$$

Trong đó, đoạn $((row \bmod 2) \times \frac{W}{2})$ tạo ra hiệu ứng so le (offset) cho các hàng lẻ.

11.3 Thuật toán xác định vị trí chuột (Inverse Mapping)

Do tính chất so le, việc dùng phép chia nguyên đơn giản là không khả thi. Hệ thống sử dụng thuật toán **Tìm kiếm lân cận gần nhất (Nearest Neighbor)**:

1. Lấy tọa độ chuột (m_x, m_y) .
2. Duyệt qua tất cả ô (r, c) trên Board.
3. Tính khoảng cách Euclidean $d = \sqrt{(m_x - x_p)^2 + (m_y - y_p)^2}$.
4. Chọn ô có d nhỏ nhất (với ngưỡng $d < 50$) làm vị trí mục tiêu.

12 Thuật toán xử lý Logic

12.1 Logic Kiểm tra Đặt vật thể (canPlace)

Hệ thống kiểm soát chặt chẽ số lượng đơn vị thông qua hàm `canPlace`.

- **Giới hạn sức chứa:** $Max = playerLevel + 1 + bonusSlots$.
- **Logic điều kiện:** Nếu một đơn vị di chuyển từ hàng dự trữ (hàng 5) vào khu vực chiến đấu (hàng 0-4), hệ thống sẽ đếm tổng số cây hiện có. Nếu đạt ngưỡng Max, hành động sẽ bị từ chối (`return false`).

12.2 Cơ chế An toàn (returnBackToSafePos)

Để tránh việc người chơi làm mất đơn vị khi thả chuột sai vị trí, thuật toán thực hiện:

1. Kiểm tra vị trí cũ của đơn vị trên Board. Nếu tìm thấy, đưa đơn vị về đó.
2. Nếu đơn vị mới hoàn toàn, tìm ô trống đầu tiên trong hàng dự trữ (hàng 5).
3. Nếu không còn chỗ trống, đối tượng sẽ bị xóa để giải phóng bộ nhớ.

13 Mã nguồn triển khai (Source Code)

Dưới đây là một phần mã nguồn triển khai logic vẽ và tính toán tọa độ:

Listing 1: Logic tính tọa độ và vẽ Hexagon

```
public int getXCoord(int col, int row) {
    double offset = (row % 2 == 1) ? HEX_W / 2.0 : 0;
    return (int) (ORIGIN_X + col * STEP_X + offset);
}

private void drawHexagon(GreenfootImage canvas, int cx, int cy) {
    int[] px = new int[6], py = new int[6];
    for (int i = 0; i < 6; i++) {
        double angle = Math.toRadians(60 * i - 90);
        px[i] = (int) (cx + HEX_R * Math.cos(angle));
        py[i] = (int) (cy + HEX_R * Math.sin(angle));
    }
    canvas.drawPolygon(px, py, 6);
}
```


14 Kết luận

Lớp `GridManager` thể hiện một cách tiếp cận chặt chẽ trong việc quản lý không gian trò chơi. Sự kết hợp giữa đại số tuyến tính cơ bản và cấu trúc dữ liệu mảng giúp hệ thống hoạt động ổn định, hiệu quả và cung cấp phản hồi hình ảnh mượt mà cho người chơi.